Scalepilled Defensemaxxers: Adaptive Reasoning-Length Allocation for Suspicious Prompts

Matthew Ho Murray Kang Haomin Qi
University of California, San Diego
Email: mah048@ucsd.edu, mukang@ucsd.edu, h5qi@ucsd.edu

Abstract

Safety guard models protect LLMs but use fixed inference-time compute and remain vulnerable to jailbreaks. Meanwhile, test-time scaling shows that allocating more reasoning compute at inference improves model robustness. This work asks: Can test-time scaling be used for safety by selectively increasing reasoning compute only for suspicious prompts? We study the reasoning-based safety guard GuardReasoner on the WildJailbreak jailbreak benchmark and systematically vary test-time compute along three axes: (i) parallel scaling via multi-sample reasoning with confidence-aware aggregation, (ii) sequential scaling via controlled reasoning extensions, and (iii) latent scaling via diffusion-style refinement. Building on these mechanisms, we design adaptive policies that trigger extra compute when the guard's own confidence is low. Experiments show that parallel scaling with DeepConf-style aggregation boosts accuracy but raises false positives, while naïve sequential scaling—even with "sharp" reasoning extensions—can hurt robustness. A single carefully prompted reasoning extension provides modest, stable robustness gains without increasing false positives. Jailbreak detection scales cleanly with latent per-token compute via diffusion across timesteps, and simple confidence-triggered policies capture the benefits of maximum test-time compute while using far less on average. Together, these results motivate treating reasoning allocation as an explicit security control in LLM safety pipelines.

1. Introduction

Long reasoning models display a fundamental tension between safety, robustness, and inference-time cost. Increasing compute at test time can improve adversarial robustness in controlled settings, but longer chains of thought can also enlarge the attack surface and expose intermediate reasoning that can be manipulated or exfiltrated [4, 9, 12, 13]. Existing safety pipelines typically rely on guard models that classify prompts or responses as safe or unsafe and either block or rewrite questionable inputs. Their compute budget is fixed and independent of prompt difficulty, and uncertain cases are usually handled by conservative blocking.

This project asks how to allocate inference-time reasoning adaptively for safety guards. Instead of executing a guard with a fixed budget for all inputs, we seek policies that spend more compute only on *suspicious* or *ambiguous* prompts. Intuitively, the system should reason more deeply when confidence is low and avoid unnecessary computation for clearly benign queries. The goal is to explore when and how test-time scaling can improve robustness in realistic jailbreak settings without introducing avoidable risk.

2. Related Work

2.1 Jailbreaking and WildTeaming

Jailbreaking denotes a class of attacks where an adversary crafts prompts that induce an aligned LLM to produce harmful content despite safety training. Recent work shows that universal adversarial suffixes can transfer across models and APIs [14], and that in-the-wild interaction logs contain a rich diversity of jailbreak tactics.

WildTeaming introduces an automatic red-teaming framework that mines real user–chatbot interactions and synthesizes additional adversarial prompts [5]. The resulting WildJailbreak dataset contains both direct harmful queries and complex multi-step jailbreaks, as well as challenging benign inputs that resemble attacks superficially. This setting is a natural testbed for safety guards and for studying compute-aware defenses.

2.2 Guard models and long reasoning

Guard models are auxiliary classifiers that operate on prompts and/or responses to detect policy violations before or after the main LLM generates a reply. Llama Guard is an early and widely deployed example of an LLM-based input–output safeguard for conversational assistants [3]. More recent work augments guard models with explicit reasoning: GuardReasoner trains a safety model to produce detailed chains of thought together with safety labels, achieving stronger performance and better interpretability than label-only baselines [7]. ThinkGuard further incorporates structured critiques to strengthen caution and macro-level robustness [11].

Parallel work analyzes the security properties of long reasoning. Zaremba et al. show that test-time budget increases can improve adversarial robustness when the internal reasoning tokens are hidden from the attacker [13]. SafeChain and follow-up work demonstrate that exposing long chains of thought changes the threat model: longer reasoning can both help and hurt safety by providing additional hooks for adversarial steering [4, 12]. A recent survey argues for explicit, risk-aware policies governing how much reasoning compute to allocate under different conditions [9].

2.3 Test-time scaling and confidence aggregation

Test-time scaling refers to using additional compute at inference to improve performance without changing the base model parameters. Self-Consistency proposes sampling multiple reasoning paths and selecting the most consistent answer across samples [10]. This corresponds to parallel scaling: as the number of samples increases, accuracy improves on many reasoning tasks.

DeepConf refines this idea by estimating confidence scores for each sampled reasoning trace and filtering or reweighting them accordingly [2]. Low-confidence traces are discarded, and the final decision aggregates high-confidence traces, enabling better performance–cost trade-offs and early stopping.

The S1 framework studies sequential scaling for reasoning tasks. Budget forcing allows the model to generate longer chains by injecting special tokens or prompts that encourage continued reasoning, or to shorten chains by forcibly truncating them [8]. S1 demonstrates that modest increases in reasoning length can substantially improve performance for finetuned models.

Finally, diffusion-based finetuning offers a latent test-time scaling mechanism. Large Language Models to Diffusion Finetuning introduces an additional diffusion module operating on token embeddings; increasing the number of diffusion steps at inference monotonically improves accuracy on a range of tasks [1].

2.4 Adaptive routing and safety

Adaptive routing is a complementary line of work that changes *which* model runs, rather than how much a given model thinks. SafeRoute trains a binary router that forwards easy examples to a small safety model and sends difficult examples to a larger, more accurate guard [6]. This improves the cost–accuracy trade-off relative to using the large guard alone.

Our work focuses instead on allocating reasoning compute for a *single* guard model. However, the objectives are aligned: both adaptive routing and adaptive test-time scaling exploit heterogeneity in input difficulty to concentrate resources where they provide the most benefit.

3. Methodology

3.1 Task and evaluation metrics

We treat safety guarding on WildJailbreak as a binary classification problem. Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ denote the evaluation set, where x_i is a user query and $y_i \in \{0,1\}$ is the ground-truth label. We use $y_i = 1$ to denote a harmful query that should be blocked and $y_i = 0$ to denote a benign query that should be allowed.

The guard $f_\theta : x \mapsto \hat{y} \in \{0,1\}$ outputs a label (1=harmful), trace z, and confidence c. Using standard counts of True Positive (TP), False Positive (FP), True Positive (TN), and False Positive (FN), we report Accuracy ($\frac{\text{TP+TN}}{N}$), Precision ($\frac{\text{TP}}{\text{TP+FP}}$), Recall ($\frac{\text{TP}}{\text{TP+FN}}$), F1, and FPR ($\frac{\text{FP}}{\text{FP+TN}}$).

3.2 Base guard model and decoding

We study GuardReasoner and its 8B base model, Llama-3.1-8B-Instruct for our experiments. GuardReasoner is a reasoning-based safeguard trained to produce a natural-language explanation followed by a safety label [7]. At decoding time we configure GuardReasoner with standard sampling parameters suitable for safety classification: nucleus sampling with moderate temperature, a maximum reasoning length, and an instruction that the final line must contain a discrete label drawn from a fixed set corresponding to safe and unsafe categories.

The baseline configuration, denoted *single-pass*, calls GuardReasoner once per input and takes the resulting label as the decision. This corresponds to fixed test-time compute with no scaling.

3.3 Parallel scaling with confidence aggregation

Parallel scaling samples multiple independent reasoning traces per query and aggregates their labels. For a given input x, we obtain k samples $(z_1, \hat{y}_1), (z_2, \hat{y}_2), \ldots, (z_k, \hat{y}_k)$, where each z_j is a reasoning trace and $\hat{y}_j \in \{0,1\}$ is the associated label.

Majority aggregation. The simplest aggregator is the Self-Consistency majority vote [10]:

$$\hat{y}^{\text{maj}} = \arg\max_{y \in \{0,1\}} \sum_{j=1}^{k} \mathbf{1}[\hat{y}_j = y].$$

Ties are broken in favor of the harmful class, reflecting a conservative security posture.

DeepConf-style aggregation. Following DeepConf [2], we introduce a per-trace confidence score $c_j \in [0,1]$ extracted from the model's internal logits. Concretely, we aggregate token-level logits over the final segment of the reasoning trace to obtain a scalar margin between the chosen label and its alternative, then normalize to $[0,1]$. For each trace we retain both $(\hat{y}_j, c_j)$.

We define a filtering threshold $\tau_{\text{filter}} \in [0,1]$ and discard traces with $c_j < \tau_{\text{filter}}$. The remaining traces are combined by a confidence-weighted vote:

$$\hat{y}^{\text{deep}} = \arg\max_{y \in \{0,1\}} \sum_{j : c_j \geq \tau_{\text{filter}}} c_j \cdot \mathbf{1}[\hat{y}_j = y].$$

If all traces are filtered out, we fall back to the single-pass prediction.

Compute model. For parallel scaling we model inference cost as proportional to the number of guard invocations. Let C_{base} denote the average cost of a single GuardReasoner call (measured, for example, in generated tokens). The expected compute of a configuration with fixed k is $C_{\text{parallel}}(k) \approx k \cdot C_{\text{base}}$. For adaptive policies that vary k per input, we report the empirical average number of samples $\mathbb{E}[k(x)]$ across the evaluation set as a measure of cost.

3.4 Sequential scaling via reasoning extensions

Sequential scaling extends a single reasoning trajectory in multiple stages. For each query x we first obtain an initial trace $z^{(0)}$ and label $\hat{y}^{(0)}$. We then allow the guard to refine its reasoning and possibly revise its decision.

Reasoning Fine-Tuning. Since Llama-3.1-8B-Instruct is not inherently a reasoning model, we first need to prime it for long chain-of-thought reasoning ability. We adapt the S1 recipe of distilling from the thinking traces of a Large Reasoning Model on a set of difficult, diverse, and high quality problems for the adversarial prompt domain. In particular, we select "adversarial" examples from the WildJailbreak training split and follow the S1 procedure of filtering with length-biased weights (to favour harder problems corresponding with longer traces). We use `gpt-oss-20b` as the teacher reasoning model for this purpose and LoRA fine-tuning due to cost constraints. Otherwise, we follow training hyperparameters exactly.

Budget forcing. Budget forcing is an S1-inspired mechanism that adjusts reasoning length by prompting the model to continue thinking even when it would otherwise terminate [8]. After the initial pass, we prompt our fine-tuned model with its own reasoning and label, together with a generic instruction to reconsider its decision. Repeating this extension E times produces a sequence of traces and labels $(z^{(1)}, \hat{y}^{(1)}), \ldots, (z^{(E)}, \hat{y}^{(E)})$. The final label $\hat{y}^{(E)}$ is taken as the decision. The effective compute scales approximately linearly with E.

Safety-focused extension prompt. With negative original results following S1's "wait" extension phrase, we also explore a less "sharp" alternative "Let's keep thinking" as a way to avoid overly biasing the model towards flipping its classification verdict upon extension.

Compute model. For sequential configurations we quantify compute using the average number of generated tokens per query. If $T^{(e)}(x)$ is the number of tokens generated in extension e for input x, the average cost is $C_{\text{seq}}(E) = \frac{1}{N}\sum_{i=1}^{N}\sum_{e=0}^{E} T^{(e)}(x_i)$. We report both C_{seq} and classification metrics.

3.5 Latent scaling with diffusion refinement

Latent scaling modifies the base model's architecture with a diffusion "pathway" implemented as a set of adapters that perform cross-attention with the original attention layers of the model. Repeated iterations through this pathway acts as denoising steps, refining a continuous token embedding prediction [1]. Let e denote a token embedding produced by the frozen base model. The diffusion adapter applies T denoising steps

$$e^{(0)} \sim \mathcal{N}(e, \sigma^2 I), \quad e^{(t+1)} = g_\phi(e^{(t)}, t),$$

where g_ϕ is a learned denoising network. After T steps, $e^{(T)}$ is decoded into a token. Increasing T raises compute but typically improves accuracy.

In this project, we conceptually align our per-token scaling analysis with the diffusion framework: we consider guard checkpoints augmented with a diffusion adapter and vary T at inference, measuring the impact on safety metrics. This axis is orthogonal to parallel and sequential scaling and changes the internal refinement of representations rather than the number or length of reasoning traces.

We follow the exact architecture and fine-tuning procedures as introduced in [1] over the Llama-3.1-8B-Instruct base model.

For this scaling dimension, our compute model takes a simplified approach with the number of inference steps T proportional to total cost. While not perfect for linear comparison, it accurately captures ordering relationships.

3.6 Adaptive compute policies

A fixed scaling parameter (such as k or E) treats all inputs equally. However, many prompts are straightforward to classify, and spending extra compute on them yields negligible benefit. We therefore consider simple adaptive policies that allocate compute based on guard confidence.

Two-level confidence-triggered policy. The simplest policy operates with two budgets, k_{low} and k_{high}, and a confidence threshold $\tau \in [0,1]$.

1. Run parallel scaling with budget k_{low} and obtain a label $\hat{y}$ and a DeepConf-style confidence score $C(x)$.
2. If $C(x) \geq \tau$, accept $\hat{y}$.
3. If $C(x) < \tau$, resample with budget k_{high} and output the high-budget prediction.

This policy concentrates high-budget evaluations on low-confidence prompts.

Multi-stage policy. We also consider a multi-stage variant with a sequence of budgets $k_1 < k_2 < \cdots < k_M$ and corresponding thresholds $\tau_1 \leq \tau_2 \leq \cdots \leq \tau_M$. Starting from k_1, the policy increases k whenever confidence remains below the current threshold, up to a maximum budget k_M. Our measure of total compute sums every budget-level attempted for each prompt.

4. Experimental Results

All experiments are conducted on a fixed subset of WildJailbreak following the evaluation protocol implemented in our project code. We analyze three orthogonal test-time scaling axes for the guard model GuardReasoner—parallel scaling, sequential scaling, and latent scaling—and then study adaptive compute-allocation policies that selectively increase reasoning for low-confidence inputs. Unless otherwise noted, the positive class corresponds to harmful queries that should be blocked.

4.1 Parallel scaling

We first vary the number of independent reasoning samples k per query and aggregate the resulting labels either by simple majority voting (Self-Consistency) or by DeepConf-style confidence filtering and weighted aggregation. The corresponding quantitative trend is summarized in Table 1.

Two systematic patterns emerge from Table 1. First, for both aggregation schemes, increasing k substantially improves recall, and F1 and accuracy increase up to moderate budgets, albeit at the cost of a higher false-positive rate. This confirms that additional parallel samples enable the guard to catch more harmful prompts, but also make it more aggressive on borderline benign queries. Second, at every value of k, DeepConf-style aggregation dominates simple majority voting: it consistently achieves higher F1 and accuracy while controlling FPR more effectively. Filtering low-confidence traces and reweighting by confidence therefore makes significantly better use of the extra test-time compute than unweighted majority voting.

4.2 Sequential Scaling with Budget Forcing

We next consider sequential test-time scaling, in which a single reasoning trajectory is extended several times. Starting from an initial pass, we apply a generic "budget forcing" instruction that simply asks the model to continue thinking and possibly reconsider its decision, without emphasizing safety.

Table 2 reports the quantitative effects of this budget forcing as we increase the number of extensions from 0 to 3.

The initial single-pass configuration (Ext = 0) already offers a strong precision–recall profile. After a single generic extension, the false-positive rate jumps to around 0.5, and accuracy drops accordingly despite only modest gains in recall. Further extensions partially recover F1 but remain unstable in both performance and cost. In short, naïvely forcing the model to "think longer" does not reliably improve safety and can substantially hurt usability.

Figure 1 re-plots these metrics as curves over the number of extensions, using the "Sequential Scaling Trend". The curves make the instability of generic budget forcing particularly clear: the accuracy of the classifier seems to oscillate when incrementing the number of reasoning extensions.

A small scale qualitative analysis of the model outputs found an emerging pattern: the model tries to close its "thinking" phase after making a classification decision, but by appending "wait" to the trace and continuing sampling often results in an immediate flip of the classification. We hypothesize this is because this exact phrase was not explicitly targeted in the synthetic training dataset, making "wait" out-of-distribution for the model. Figure 1's oscillating trend seems to suggest that the parity of the number of extensions matters because it flips the binary decision back towards the initial in-domain prediction.

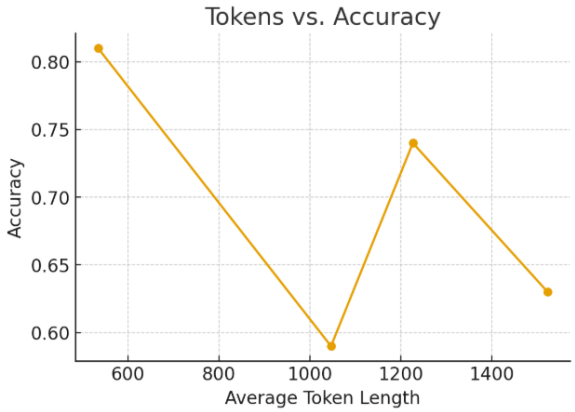

Figure 1: Sequential scaling with generic budget forcing.

4.3 Sequential scaling with safety-focused "keep thinking"

To stabilize sequential scaling, we adapt the extension instruction to be less "sharp", meaning less suggestive of being previously wrong. The new instruction ("Let's keep thinking...") is more neutral and makes no comment about the previous correctness.

As shown in Table 3, the first extension achieves a small but robust improvement over the baseline: F1 and accuracy increase, recall improves slightly, and the false-positive rate drops to zero on this subset. A second extension essentially returns the metrics to their original values, suggesting that even with a more neutral joining phrase, the model is still prone to re-thinking. Further experiments with updated synthetic training data is necessary to be more conclusive on whether sequential scaling can be simply implemented for binary classification tasks or if the small decision space is incompatible with this over-thinking approach.

4.4 Latent scaling

Latent scaling varies the number of diffusion refinement steps T in a diffusion-augmented guard, following the Large Language Models to Diffusion Finetuning framework [1]. Intuitively, more diffusion steps correspond to more computation spent denoising the internal representation of the input before decoding a label.

In our experiments, we sweep several values of T and measure accuracy and F1 on the same WildJailbreak subset. We find that accuracy improves rapidly when moving from very small to moderate T, then saturates as further steps yield diminishing gains. This monotone but saturating behavior mirrors the original diffusion-finetuning results and suggests that latent scaling can be tuned to balance robustness against cost without changing the guard's external interface.

The corresponding trend is summarized in Table 4, which reports F1 and accuracy on WildJailbreak as a function of the number of diffusion inference steps.

Table 1: Parallel scaling with k reasoning samples for GuardReasoner on WildJailbreak. The left block uses simple majority voting (Self-Consistency); the right block uses DeepConf-style confidence filtering and confidence-weighted aggregation.

	Majority aggregation (Self-Consistency)					DeepConf-style aggregation				
k	Precision	Recall	F1	Accuracy	FPR	Precision	Recall	F1	Accuracy	FPR
1	0.701	0.550	0.667	0.621	0.15	0.727	0.561	0.639	0.666	0.13
2	0.682	0.652	0.667	0.663	0.20	0.712	0.683	0.691	0.701	0.17
3	0.664	0.721	0.667	0.690	0.25	0.701	0.758	0.720	0.720	0.20
4	0.641	0.751	0.667	0.690	0.28	0.693	0.782	0.734	0.740	0.21
5	0.637	0.779	0.667	0.690	0.30	0.685	0.806	0.743	0.750	0.22
6	0.622	0.782	0.667	0.690	0.31	0.680	0.817	0.748	0.750	0.22
7	0.620	0.790	0.667	0.690	0.32	0.680	0.826	0.752	0.760	0.24
8	0.613	0.811	0.667	0.690	0.33	0.673	0.835	0.759	0.760	0.25

Table 2: Sequential scaling with generic budget forcing. "Ext" is the number of additional reasoning extensions beyond the initial pass. "Tokens" is the mean number of generated tokens per query.

Ext	Tokens	Prec	Rec	F1	Acc	FPR
0	534.35	0.970	0.64	0.771	0.81	0.02
1	1046.77	0.576	0.68	0.624	0.59	0.50
2	1226.76	0.745	0.70	0.722	0.74	0.24
3	1523.28	0.614	0.70	0.654	0.63	0.44

Table 3: Sequential scaling with a safety-focused "keep thinking" extension.

Ext	Tokens	Prec	Rec	F1	Acc	FPR
0	534.35	0.970	0.64	0.771	0.81	0.02
1	907.99	1.000	0.66	0.795	0.83	0.00
2	1393.60	0.970	0.64	0.771	0.81	0.02

4.5 Adaptive compute allocation

Finally, we study adaptive regimes that allocate additional compute only to low-confidence inputs. The guard produces a DeepConf-style confidence estimate alongside its prediction, and this estimate is used to decide whether to trigger higher budgets.

Basic two-level policy. The basic policy maintains two budgets k_{low} and k_{high} for parallel scaling. For each query, we first run the diffusion tuned model with budget k_{low} and obtain a confidence score. If the confidence exceeds a threshold τ, we accept the low-budget decision; otherwise, we re-evaluate with budget k_{high}.

The "Adaptive Regime (Basic Policy)" figure plots accuracy and cost score (proportional to T) for different values of the confidence threshold τ. A key operating point is $\tau = 0.8$, at which the policy recovers the maximum accuracy achievable by always running at high budget while using only about 63% of the maximum compute. This demonstrates that simple confidence-triggered routing can recover most of the benefit

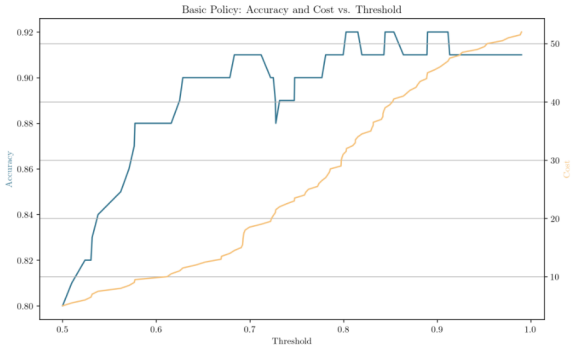

Figure 2: Adaptive regime with a basic two-level policy. The curve shows accuracy versus compute for different confidence thresholds τ. The highlighted point at $\tau = 0.8$ achieves near-maximal performance at roughly 63% of the maximum cost.

of aggressive test-time scaling at substantially lower average cost. Notably, at various points with this basic policy, we achieve an accuracy of 92% which is even greater than the static "only-max-compute" approach which gets 91%. This indicates that considering high confidence predictions at lower computes can sometimes even outperform the max compute prediction.

Multi-step policy. We additionally explore a multi-step adaptive regime with several intermediate budgets and thresholds. Intuitively, the policy starts with a small budget and increases k in stages whenever the current confidence remains below the corresponding threshold, up to a maximum budget. This allows the system to spend very little compute on clearly benign or clearly harmful prompts, while allocating much higher budgets to genuinely ambiguous cases.

The "Adaptive Regime (Multi-Step Policy)" figure visualizes the trade-off surface between average compute and accuracy for this multi-stage design as a function of the confidence threshold. The resulting frontier is smooth: as thresholds become more conservative, the policy approaches the full high-budget configuration; as they become more permissive, the average compute drops while accuracy degrades gracefully. The multi-step policy therefore offers additional

Table 4: Latent scaling with a diffusion adapter: F1 and accuracy on WildJailbreak (Wild Jailbreak subset) as a function of the number of diffusion inference steps.

Methods	# Inference Steps	F1 (Wild Jailbreak)	Accuracy (Wild Jailbreak)
Standard SFT (Full 8B)	1	0.76	0.78
Standard SFT LoRA (1B Params)	1	0.67	0.65
Diffusion Finetuning (281M)	1	0.77	0.78
Diffusion Finetuning (281M)	5	0.79	0.80
Diffusion Finetuning (281M)	10	0.82	0.84
Diffusion Finetuning (281M)	25	0.83	0.87
Diffusion Finetuning (281M)	50	0.85	0.91

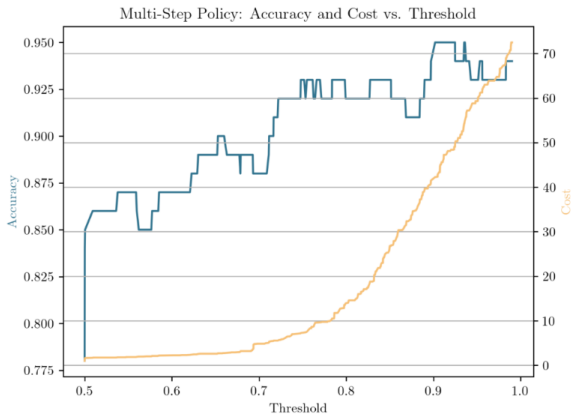

Figure 3: Adaptive regime with a multi-step policy, illustrating how accuracy scales with compute under multi-stage confidence thresholds and budgets.

flexibility for tuning the cost–robustness operating point.

Again, we find a similar but even more pronounced trend compared to the Basic Policy– the maximum accuracy of this setup is 0.95 which further outstrips that of the static max-cost inference (0.91). Again this maximum occurs before the maximum spend as seen in figure 3. We can formalize this as a slightly more principled selection of the threshold: given a specified tolerance ε representing how much accuracy we are willing to sacrifice, find the threshold τ that minimizes cost while adhering to this accuracy constraint. In the multi-step regime, we find threshold 0.902 to be within 1% of the max accuracy (this is, in fact, the max, so no accuracy degradation) while using around 60% of the max cost.

4.6 Discussion and Limitations

The experimental study leads to several conclusions relevant to the design of safety guards for LLMs.

First, test-time scaling is not uniformly beneficial. Parallel scaling with confidence-aware aggregation consistently improves robustness, whereas generic sequential scaling via budget forcing can sharply increase false positives and degrade overall accuracy. Sequential scaling becomes reliable only when the extension prompt is explicitly tailored to safety.

Second, confidence signals derived from the guard's own reasoning are highly informative. They not only enable DeepConf-style filtering and weighting for parallel aggregation but also support simple adaptive routing policies that recover most of the benefits of maximum compute with substantially reduced average cost.

Third, latent scaling provides an orthogonal knob: by adjusting the number of diffusion refinement steps, one can trade additional internal compute for modest gains in robustness without changing the model's outer behavior. This axis is particularly appealing for deployment scenarios where external chain-of-thought exposure is constrained.

The main limitations are scope and calibration. All results are obtained on a single dataset and guard model; generalization to other domains, languages, or safety taxonomies remains to be established. Our confidence estimators and thresholds are deliberately simple; more principled uncertainty estimation or risk calibration could further improve adaptive policies. Finally, while we have focused on classification performance, a full security analysis must also account for how longer or more intensive reasoning affects the attack surface of the guard itself, especially under chain-of-thought exposure [4, 9, 12].

5. Conclusion

This project investigates adaptive reasoning-length allocation for safety guards under jailbreak attacks. By first studying and characterizing scaling trends with parallel, sequential, and latent test-time scaling, and then simple routing policies, we demonstrate that additional compute can be directed toward suspicious prompts in a controlled way. We find clean scaling relations in the parallel and latent dimensions with more mixed results with sequential scaling. Our confidence-gated policies recover most if not all the benefit of increased compute with substantially lower average cost.

From a security perspective, the key insight is that inference-time compute is not merely an efficiency parameter but also a security parameter. How much and where a safety guard is allowed to think should be treated as a first-class design choice, grounded in empirical evidence and integrated with broader risk assessments. Future work should extend this evaluation to diverse datasets and guard architectures, incorporate more principled uncertainty estimates, and develop adversarial analyses of long-reasoning defenses.

Appendix: Team Member Contributions

Matthew Ho. Led the design and implementation of the sequential scaling experiments, including both the generic budget forcing and the safety-focused "keep thinking" extensions. Conducted the quantitative evaluation, qualitative case study analysis, and drafted the text for the sequential scaling subsections. Additionally, designed and ran the adaptive compute experiments and drafted corresponding visualizations and discussion.

Murray Kang. Led the design and implementation of the latent scaling experiments with diffusion finetuning, including configuration of inference steps, evaluation on the Wild Jailbreak subset, and preparation of the latent scaling table and analysis.

Haomin Qi. Led the design and implementation of the parallel scaling experiments, including the Self-Consistency and DeepConf-style aggregation setups, data processing, metric computation, and analysis. Prepared the corresponding tables and plots, and drafted the text for the parallel scaling subsection.

All authors jointly contributed to problem formulation, literature review, overall methodology design, interpretation of results, and writing and editing of the introduction, background, discussion, and conclusion sections. Contributions are approximately equal across team members.

References

[1] Cetin, E., Zhao, B., et al. Large language models to diffusion finetuning, 2025.

[2] Fu, Y., et al. Deepconf: Deep think with confidence for test-time scaling, 2025.

[3] Inan, H., Upasani, K., Chi, J., Rungta, R., Iyer, K., Mao, Y., Tontchev, M., Hu, Q., Fuller, B., Testuggine, D., and Khabsa, M. Llama guard: Llm-based input-output safeguard for human-ai conversations, 2023.

[4] Jiang, F., Xu, Z., Li, Y., Niu, L., Xiang, Z., Li, B., Lin, B. Y., and Poovendran, R. Safechain: Safety of language models with long chain-of-thought reasoning capabilities, 2025.

[5] Jiang, L., Rao, K., Han, S., Ettinger, A., Brahman, F., Kumar, S., Mireshghallah, N., Lu, X., Sap, M., Choi, Y., and Dziri, N. Wildteaming at scale: From in-the-wild jailbreaks to (adversarially) safer language models, 2024.

[6] Lee, S., Lee, D. B., Wagner, D., Kang, M., Seong, H., Bocklet, T., Lee, J., and Hwang, S. J. Saferoute: Adaptive model selection for efficient and accurate safety guardrails in large language models. In *Findings of the Association for Computational Linguistics: ACL 2025* (Vienna, Austria, 2025), Association for Computational Linguistics, pp. 2053–2069.

[7] Liu, Y., Gao, M., Chen, X., et al. Guardreasoner: Towards reasoning-based llm safeguards, 2025.

[8] Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., and Hashimoto, T. s1: Simple test-time scaling, 2025.

[9] Wang, C., Liu, Y., Bi, B., Zhang, D., Li, Z.-Z., Ma, Y., He, Y., Yu, S., Li, X., Fang, J., Zhang, J., and Hooi, B. Safety in large reasoning models: A survey, 2025.

[10] Wang, X., Wei, J., Schuurmans, D., Le, Q. V., Chi, E. H., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models, 2022.

[11] Wen, X., Zhou, W., Mo, W., and Chen, M. Thinkguard: Deliberative slow thinking leads to cautious guardrails, 2025.

[12] Wu, T., Xiang, C., Wang, J. T., Yu, W., Sitawarin, C., Sehwag, V., and Mittal, P. Does more inference-time compute really help robustness?, 2025.

[13] Zaremba, W., Nitishinskaya, E., Barak, B., Lin, S., Toyer, S., Yu, Y., Dias, R., Wallace, E., Xiao, K., Heidecke, J., and Glaese, A. Trading inference-time compute for adversarial robustness, 2025.

[14] Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., and Fredrikson, M. Universal and transferable adversarial attacks on aligned language models, 2023.